SOFTWARE REQUIREMENTS SPECIFICATION

Pharma Availability Control Tower

7-Day MVP — Planner-Facing Dispatch Console

Version 2.0 | 2026

Status: Draft — Pre-Engine Integration

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the Pharma Availability Control Tower MVP. The document covers platform infrastructure, snapshot ingestion, decision engines M1/M2/M3, the planner console, and the demo-state simulation module.

The intended audience includes developers, architects, QA engineers, and project stakeholders.

1.2 Project Overview

The Control Tower is a planner-facing pharma dispatch decision system. It ingests operational snapshots, runs three decision engines, and presents a unified console for a human planner to review, edit, approve, or override dispatch plans. A separate demo-state module simulates stock movement post-approval for demonstration purposes.

1.3 Scope

In-scope for this MVP:

- Snapshot ingestion from six reader types
- ETA refresh from a mock API provider
- M1: Medicine Priority Tracker — shipment-SKU priority scoring
- M2: DC Request Generator — demand-driven restocking requests
- M3: Dispatch Planner — optimized candidate plan generation and ranking
- Planner console: dashboard, inputs, engine outputs, approval flow, history, reports
- Demo-state simulation: reservation creation and simulated stock movement
- Contract-compatible stubs for M1, M2, M3 to unblock platform and frontend work

Out of scope:

- Live ERP or WMS integration
- Multi-warehouse or multi-region scenarios beyond the locked MVP fleet
- Mobile clients or external-facing portals
- Real-time event streaming

1.4 Definitions and Acronyms

Term	Definition
SKU	Stock-Keeping Unit — uniquely identifies a medicine variant
DC	Distribution Centre — regional fulfilment node
M1	Medicine Priority Tracker engine — scores shipment-SKU lines
M2	DC Request Generator engine — produces restocking requests

M3	Dispatch Planner engine — generates and ranks candidate plans
Snapshot	Immutable, timestamped read of an operational data source
Demo-state	Isolated simulation module that shows post-approval stock progression
Stub	Contract-compatible placeholder that mimics a real engine's output shape
ETA	Estimated Time of Arrival for a vessel
ORM	Object-Relational Mapper — SQLAlchemy 2.x is used in this project
DSN	Data Source Name — MySQL connection string for the application
Effective WH Stock	Effective Warehouse Stock — Physical DB Stock minus Reserved DB Stock; the inventory value passed to all decision engines
Effective DC Stock	Effective DC Stock — Physical DB DC Stock plus In-Transit DB DC Stock; the inventory value passed to all decision engines
Math-Bound Validation	Pure math check applied to planner overrides before plan freezing: capacity, Effective Warehouse Stock limits, and reefer compatibility rules

1.5 Document Conventions

- SHALL — mandatory requirement
- SHOULD — recommended but not strictly mandatory
- MAY — optional
- FR-XXX — Functional Requirement identifier
- NFR-XXX — Non-Functional Requirement identifier
- UC-XXX — Use Case identifier

1.6 References

- PLATFORM_DEMO_STATE_PLAN.md — detailed platform delivery steps
- PROJECT_PLAN_7_DAY_MVP.md — day-by-day delivery schedule
- README.md — Pharma Availability Control Tower overview
- STRATEGY.md — architectural strategy and decisions

2. System Overview

2.1 System Context

The Control Tower sits between operational data sources (vessel manifests, warehouse stock, DC stock, sales history, lorry state, ETA feeds) and a human planner. It does not own or mutate operational stock. Its role is to present timely, prioritised information and support the planner in making and recording dispatch decisions.

2.2 High-Level Architecture

The system is composed of four cleanly separated layers:

Layer	Responsibility
Ingestion	Snapshot readers and ETA provider fetch and persist operational data
Engines	M1, M2, M3 consume snapshots and produce scores, requests, and plans — no stock mutation
Planner Console	Next.js frontend for review, editing, approval, rejection, and override
Demo State	Isolated simulation of post-approval reservation and stock movement for demo use only

2.3 Core Data Flow

1. The planner clicks “Generate Plan” in the UI, triggering all decision models on-demand. Readers immediately fetch Effective Warehouse Stock (Physical − Reserved), Effective DC Stock (Physical + In-Transit), sales history, and binary lorry availability.
2. ETA provider refreshes vessel ETA from the mock API and persists ETA snapshots.
3. M2 generates DC restocking requests from Effective DC Stock (Physical + In-Transit) and 48-hour sales forecasts, on-demand only.
4. M1 scores each shipment-SKU line using manifests, warehouse stock, M2 requests, SKU metadata, and ETA timing.
5. M3 generates and ranks candidate dispatch plans from M1 output, M2 output, warehouse stock, lorry state, and the route graph.
6. The planner reviews, edits, approves, rejects, or overrides a draft plan in the console.
7. On approval, the system pushes reservation records and in-transit allocations directly to the database, freezing the plan. Physical stock progression for the demo is driven by backend CLI scripts (simulate_vessel_arrival.py, simulate_lorry_arrival.py) rather than automated simulation triggers.
8. After CLI-simulated arrivals, the next planner “Generate Plan” action organically reflects updated physical stocks in all engine inputs.

2.4 Technology Stack

Component	Technology
Frontend	Next.js (React) — planner-only console
Backend API	FastAPI (Python) — orchestration, planner actions, reporting
Database	MySQL 8 in Docker — single local database: control_tower_mvp
ORM / Migrations	SQLAlchemy 2.x, Alembic, PyMySQL
M1 / M3 Ranker	Python, XGBoost, scikit-learn
M3 Optimizer	Google OR-Tools

3. Stakeholders and Users

3.1 Stakeholders

Stakeholder	Interest	Priority
Planner	Review, edit, approve, and override dispatch plans	Critical
Supply Chain Manager	Visibility into prioritised shipments and DC stock status	High
Development Team	Clear contracts, reproducible local setup, and clean boundaries	High

3.2 User Roles

The MVP has one user role: Planner. All console features are accessible to this role. There is no authentication scope differentiation in the MVP.

4. Functional Requirements

4.1 Snapshot Ingestion

4.1.1 Manifest Reader (FR-ING-01)

- The system SHALL fetch vessel manifests from the source database reader.
- The reader SHALL support fetching the latest snapshot and a historical snapshot by ID or timestamp.
- The reader SHALL normalise raw rows into an engine-ready manifest contract.
- Manifests SHALL NOT be manually uploaded — they must come from the reader.

4.1.2 Warehouse Stock Reader (FR-ING-02)

- The system SHALL fetch warehouse stock and calculate Effective Warehouse Stock presented to all prediction models as: Effective Warehouse Stock = Physical DB Stock − Reserved DB Stock.
- The reader SHALL support latest and historical snapshot retrieval.
- The reader SHALL normalise rows into engine-ready warehouse stock contracts and execute immediately on planner “Generate Plan” triggers only — no background polling.

4.1.3 DC Stock Reader (FR-ING-03)

- The system SHALL fetch DC stock and calculate Effective DC Stock presented to all prediction models as: Effective DC Stock = Physical DB DC Stock + In-Transit DB DC Stock. The reader SHALL cover all 5 DCs.
- The reader SHALL support latest and historical snapshot retrieval.

4.1.4 Sales History Reader (FR-ING-04)

- The system SHALL fetch sales history records as snapshots for M2 consumption.

4.1.5 Lorry State Reader (FR-ING-05)

- The system SHALL fetch lorry state snapshots covering all 8 lorries (5 normal, 3 reefer).
- Lorry availability SHALL be a binary DB state: AVAILABLE or UNAVAILABLE. Complex location tracking is out of scope for the MVP.

4.1.6 ETA Provider (FR-ING-06)

- The system SHALL fetch vessel ETA from the mock API.
- The ETA provider SHALL be the only external mock API feed in the MVP.
- Every ETA fetch SHALL be persisted as an ETA snapshot.
- The ETA provider SHALL support manual refresh and a scheduled trigger.
- ETA SHALL refresh at least every 2 hours during demo operation.

4.2 Decision Engines

4.2.1 M1 — Medicine Priority Tracker (FR-ENG-01)

- M1 SHALL accept manifest lines, Effective Warehouse Stock (Physical − Reserved), active M2 requests, SKU metadata, and ETA-derived timing as inputs. M1 SHALL execute strictly on-demand via the planner “Generate Plan” action — no background polling.
- M1 SHALL produce a priority_score and priority_band per shipment-SKU line.
- M1 SHALL produce a reefer_required flag per line.
- M1 SHALL produce an aggregated SKU-level summary for dashboard display.
- M1 SHALL never mutate any stock data.
- M1 SHALL use XGBoost for priority scoring.

4.2.2 M2 — DC Request Generator (FR-ENG-02)

- M2 SHALL accept Effective DC Stock (Physical + In-Transit) and 48-hour sales forecasts as inputs. M2 SHALL execute strictly on-demand via the planner “Generate Plan” action — no background polling or daily scheduling.
- M2 SHALL produce dc_request, urgency, and required_by fields per request.
- M2 SHALL generate requests covering a 48-hour forward planning horizon, executed only when triggered by the planner.
- M2 SHALL never mutate any stock data.

4.2.3 M3 — Dispatch Planner (FR-ENG-03)

- M3 SHALL accept M1 output, M2 output, Effective Warehouse Stock, binary lorry state snapshot (AVAILABLE/UNAVAILABLE), and the route graph as inputs. M3 SHALL plan for a strict 48-hour forward horizon. M3 SHALL generate a maximum of 1 route (trip) per available lorry per planning generation cycle. M3 SHALL execute strictly on-demand via the planner “Generate Plan” action.
- M3 SHALL generate feasible candidate dispatch plans using Google OR-Tools.
- M3 SHALL rank candidate plans using the XGBoost-based ranker.
- M3 SHALL expose a planner-editable draft plan.
- M3 SHALL respect a maximum of 2 stops per lorry.
- M3 SHALL enforce reefer cargo on reefer lorries only.
- M3 SHALL enforce normal cargo on normal lorries only.
- M3 SHALL use capacity_unit for lorry capacity and load feasibility checks.
- M3 SHALL never mutate any stock data.

4.2.4 Engine Stubs (FR-ENG-04)

- Contract-compatible stubs SHALL be provided for M1, M2, and M3.
- Stubs SHALL return responses in the identical shape to the real engines.
- Stubs SHALL use deterministic fixtures to support frontend and planner flow development.
- A clean switch SHALL exist so stubs can be replaced by real engines without changing API routes or frontend screens.

4.3 Planner Console

4.3.1 Dashboard (FR-UI-01)

- The dashboard SHALL display alerts, latest engine run summaries, and pending approvals.
- The dashboard SHALL show the aggregated M1 SKU priority summary.

4.3.2 Inputs View (FR-UI-02)

- The inputs view SHALL display read-only manifests, ETA, warehouse stock, DC stock, sales history, and lorry snapshots.

4.3.3 M1 Priorities View (FR-UI-03)

- The view SHALL display shipment-line priority scores and priority bands.
- The view SHALL also display the aggregated SKU-level summary.

4.3.4 M2 Requests View (FR-UI-04)

- The view SHALL display generated DC requests with urgency and required_by fields.

4.3.5 M3 Dispatch View (FR-UI-05)

- The view SHALL display candidate plans with ranking scores.
- The planner SHALL be able to compare candidate plans.
- The planner SHALL be able to edit lorry choice, stop order, and quantities on a draft plan before approval. When a planner overrides (alters) a generated draft plan, the system SHALL validate the manual changes via a Math-Bound Validation step before allowing the plan to freeze. Validation is restricted to pure math bounds: (1) total shipment quantity cannot exceed lorry capacity, (2) total SKU quantity cannot exceed Effective Warehouse Stock (Physical − Reserved), and (3) reefer lorries must be assigned to reefer SKUs only.
- The planner SHALL be able to approve, reject, or override a plan.
- Once a plan version is approved, it SHALL be permanently frozen and immutable. Upon approval, the system SHALL immediately push reservation records (Warehouse) and in-transit allocation records (DC) directly to the primary database, preventing ghost inventory double-allocation in future planning cycles.

4.3.6 History View (FR-UI-06)

- The history view SHALL display all approved plan versions.
- The view SHALL show the override trail for plans that replaced prior versions.

4.3.7 Demo State View (FR-UI-07)

- The view SHALL display reservation status, in-transit transfers, and completed arrivals.
- The view SHALL display projected warehouse and DC stock levels.

4.3.8 Reports View (FR-UI-08)

- The reports view SHALL provide export views for approved plans.
- The view SHALL provide audit views tying together snapshots, engine runs, plan versions, decisions, and demo-state events.

4.4 Demo State Simulation

4.4.1 Reservation Creation (FR-DS-01)

- On planner approval of a plan version, the demo-state module SHALL create reservation records.
- Reservation records SHALL be created for each approved stop and item.

4.4.2 Transfer Records (FR-DS-02)

- The demo-state module SHALL create transfer records per approved stop and item.

4.4.3 Arrival Simulation (FR-DS-03)

- For demonstration purposes, physical state progression (vessel arrivals, lorry deliveries) SHALL be simulated via standalone backend CLI scripts interacting directly with the database, rather than through complex UI simulation dashboards or model-level internal mock states. The two required scripts are: simulate_vessel_arrival.py (reads active manifests and increments physical warehouse DB stock) and simulate_lorry_arrival.py (closes an active transfer, removes warehouse reservations, and increments physical DC DB stock).
- Running simulate_vessel_arrival.py SHALL increment physical warehouse DB stock for the relevant SKUs and drop the corresponding manifest record.
- Running simulate_lorry_arrival.py SHALL increment physical DC DB stock, remove the active warehouse reservation, and close the in-transit transfer record.
- After CLI scripts run, the next planner “Generate Plan” action SHALL automatically reflect updated Effective Warehouse Stock and Effective DC Stock in all engine inputs — no manual sync is required.

4.4.4 Demo Isolation (FR-DS-04)

- The demo-state module SHALL be entirely separate from engine logic.
- Demo-state records SHALL derive only from approved plan versions.
- M1, M2, and M3 SHALL never read from or write to demo-state tables directly.

5. Data Model

5.1 Master Tables

Table	Description
skus	15 medicine SKUs with reefer_required flag and capacity_unit metadata
dcs	5 distribution centres with location and capacity metadata
lorries	8 lorries: 5 normal, 3 reefer — with capacity_unit and lorry type
route_edges	Fixed route graph with travel time and cost between nodes
vessels	Vessel master data referenced by manifest snapshots

5.2 Snapshot Tables

Table	Description
manifest_snapshots / manifest_lines	Vessel manifest headers and line items
warehouse_stock_snapshots / items	Warehouse-level stock snapshot and per-SKU items
dc_stock_snapshots / items	Per-DC stock snapshot and per-SKU items
sales_history_records	Historical sales data used by M2
lorry_state_snapshots / items	Current lorry availability and position
eta_snapshots	ETA records persisted from every mock API fetch

5.3 Engine Run Tables

Table	Description
engine_runs	Root record for each engine invocation; stores source snapshot IDs
m1_results	Per-line priority scores, bands, and reefer flags from M1
m2_requests	DC restocking requests with urgency and required_by from M2
m3_plan_versions	Candidate and approved plan version headers with status and ranking score
m3_plan_stops	Stop sequence per plan version
m3_plan_items	SKU line items per stop per plan version

5.4 Planner Flow Tables

Table	Description

planner_decisions	Records of approve, reject, and override actions with timestamps and planner ID
override_reasons	Free-text rationale linked to planner decisions of type override

5.5 Demo State Tables

Table	Description
demo_reservations	Reservation records created on plan approval
demo_transfers	Transfer records per approved stop and item
demo_arrival_events	Simulated arrival events that trigger stock projection updates
demo_stock_projections	Projected warehouse and DC stock derived from approved plans and arrivals

5.6 Audit

- audit_logs SHALL tie together snapshots, engine runs, plan versions, planner decisions, and demo-state events.
- All timestamps SHALL be stored in UTC.
- Indexes SHALL be created on timestamps, vessel IDs, SKU IDs, DC IDs, lorry IDs, plan version IDs, and approval status.

6. Use Cases

6.1 UC-01 — Planner Reviews Prioritised Shipment Lines

Actor: Planner

Precondition: At least one M1 engine run exists.

Main Flow:

9. Planner navigates to the M1 Priorities view.
10. System displays shipment-SKU lines with priority_score, priority_band, and reefer_required.
11. System also displays the aggregated SKU summary.
12. Planner reviews and cross-references with the M2 requests view.

6.2 UC-02 — Planner Approves a Dispatch Plan

Actor: Planner

Precondition: A draft M3 plan version exists.

Main Flow:

13. Planner opens the M3 Dispatch view.
14. Planner compares candidate plans and selects one.
15. Planner optionally edits lorry choice, stop order, or quantities.
16. Planner selects Approve.
17. System freezes the plan version as immutable.
18. System creates demo reservation and transfer records.

Postcondition: Plan version is permanently frozen. Demo-state records exist.

6.4 UC-04 — Demo Arrival Simulation

Actor: Developer (CLI script execution)

Precondition: Approved plan exists with demo reservation and transfer records.

Main Flow:

19. Developer runs simulate_vessel_arrival.py or simulate_lorry_arrival.py from the command line against the local database.
20. Script physically increments warehouse DB stock (vessel arrival) or physically increments DC DB stock and closes the active transfer (lorry arrival).
21. Script removes active warehouse reservations where applicable, ensuring Effective Stock calculations are correct for the next planning cycle.
22. Next planner “Generate Plan” action organically reflects the updated physical stocks — no manual refresh required.

6.5 UC-05 — ETA Refresh

Actor: Scheduler or Planner

Precondition: Vessels exist in the master table.

Main Flow:

23. ETA provider is triggered (scheduled or manual).
24. System calls the mock ETA API.
25. Response is persisted as a new ETA snapshot.
26. Dashboard shows updated ETA values.

7. API Specification

7.1 API Route Groups

Route Prefix	Responsibility
/api/v1/inputs/*	Read snapshots: manifest, stock, sales, lorry state, ETA
/api/v1/orchestration/*	Assemble and persist engine input contracts; trigger engine runs
/api/v1/planner/*	Dashboard summary, engine result views, plan approval and override actions
/api/v1/demo-state/*	Reservation status, transfer status, projected stock views
/api/v1/reports/*	Approved plan exports, audit views
/api/v1/mock/eta/*	Manual ETA refresh endpoint for the mock provider

7.2 Key Planner Endpoints

Method	Path	Description
GET	/api/v1/planner/dashboard	Alerts, latest engine runs, pending approvals
GET	/api/v1/planner/m1/results/{run_id}	M1 line scores and SKU summary for a run
GET	/api/v1/planner/m2/requests/{run_id}	M2 DC requests for a run
GET	/api/v1/planner/m3/plans/{run_id}	All candidate plans for a run with ranking
POST	/api/v1/planner/plans/{id}/approve	Freeze plan version and trigger demo-state creation
POST	/api/v1/planner/plans/{id}/reject	Mark plan version as rejected
POST	/api/v1/planner/plans/{id}/override	Create a manual overridden draft version from an M3 candidate plan prior to approval.

8. Non-Functional Requirements

8.1 Performance (NFR-PERF)

- NFR-PERF-01: Dashboard page SHALL load within 2 seconds on a developer laptop with seeded demo data.
- NFR-PERF-02: Snapshot readers SHALL complete latest-fetch queries within 500 ms.
- NFR-PERF-03: M3 SHALL produce at least 3 candidate plans within 30 seconds for the locked MVP scope.

8.2 Reliability (NFR-REL)

- NFR-REL-01: The system SHALL support full reset and reseed of demo scenarios without manual database intervention.
- NFR-REL-02: The demo MUST be runnable end-to-end on a single developer laptop.
- NFR-REL-03: Docker volume persistence SHALL ensure MySQL data survives container restarts.

8.3 Maintainability (NFR-MAINT)

- NFR-MAINT-01: Engine stubs SHALL be replaceable with real engines without changing API routes or frontend screens.
- NFR-MAINT-02: All migrations SHALL use Alembic with business-intent naming.
- NFR-MAINT-03: Session wiring SHALL be defined in one location and reused across integrations, planner flow, reporting, and demo-state.

8.4 Security (NFR-SEC)

- NFR-SEC-01: Database credentials SHALL be supplied via environment variables only — never hardcoded.
- NFR-SEC-02: Root MySQL password and app user password SHALL be separate credentials.
- NFR-SEC-03: The API SHALL be accessible on localhost only in the MVP.

8.5 Data Integrity (NFR-DATA)

- NFR-DATA-01: Effective Warehouse Stock presented to all models SHALL be computed as Physical DB Stock minus Reserved DB Stock. Effective DC Stock presented to all models SHALL be computed as Physical DB DC Stock plus In-Transit DB DC Stock. These calculations SHALL prevent ghost inventory double-allocation across planning cycles.
- NFR-DATA-02: Approved plan versions SHALL be immutable — the system SHALL reject any attempt to mutate an approved plan.
- NFR-DATA-03: Demo-state records SHALL derive only from approved plan versions.
- NFR-DATA-04: All timestamps SHALL be stored in UTC.

- NFR-DATA-05: Foreign key constraints SHALL be enforced between snapshots, plan versions, planner decisions, and demo-state tables.

8.6 Portability (NFR-PORT)

- NFR-PORT-01: The full stack SHALL start with standard docker compose commands.
- NFR-PORT-02: MySQL character set SHALL be utf8mb4 with InnoDB engine for all tables.
- NFR-PORT-03: Application DSN SHALL use the pattern mysql+pymysql://<user>:<password>@<host>:3306/control_tower_mvp?charset=utf8mb4.

9. Constraints and Assumptions

9.1 Locked MVP Scope

Parameter	Constraint
SKUs	Exactly 15
Warehouses	Exactly 1
DCs	Exactly 5
Lorries	Exactly 8 (5 normal, 3 reefer) — binary availability state: AVAILABLE or UNAVAILABLE
Stops per lorry	Maximum 2
Planning horizon	Strict 48-hour forward horizon per planning generation cycle
Trips per lorry	Maximum 1 trip per available lorry per planning generation cycle
Route graph	Fixed — times and costs do not change during a demo run
UI users	Planner only — no multi-role access control in MVP

9.2 Assumptions

- The MySQL 8 Docker image is available in the developer environment.
- Node.js and Python 3.x runtimes are pre-installed on the developer laptop.
- The mock ETA API endpoint is available locally for MVP purposes.
- Seeded demo data is sufficient to exercise all three deterministic scenarios.
- M1, M2, and M3 will eventually replace stubs without requiring API or frontend changes.

10. Seeded Demo Scenarios

10.1 Deterministic Scenarios

Three deterministic scenarios SHALL be seedable and replayable:

Scenario A — Overlapping Vessels

- Two vessels with overlapping ETA windows arrive at the warehouse simultaneously.
- M1 must resolve priority conflicts across manifest lines from both vessels.
- M3 must produce a feasible dispatch plan covering both arrival batches.

Scenario B — Urgent DC Shortage

- One or more DCs show critically low stock for high-priority SKUs.
- M2 generates urgent requests with near-term required_by dates.
- M1 reflects elevated priority scores for the affected SKU lines.
- M3 produces a plan that expedites delivery to the shortage DC.

Scenario C — Fleet Limitation with Reefer Pressure

- Reefer SKUs require refrigerated transport but lorry availability is constrained.
- M3 must allocate the 3 reefer lorries optimally under capacity pressure.
- The planner is presented with trade-off candidate plans showing different reefer allocation strategies.

10.2 Seed Data Summary

Entity	Seed Count / Detail
SKUs	15 (mix of reefer and normal)
DCs	5 with varying stock levels per scenario
Lorries	8 (5 normal, 3 reefer) with route positions
Vessel manifests	At least 2 with overlapping ETAs for Scenario A
Warehouse stock	Per-SKU quantity seeded for each scenario
DC stock	Per-DC, per-SKU — critical shortage seeded for Scenario B
Sales history	Rolling 30-day history to drive M2 demand signals
Route graph	Fixed edges between warehouse, DCs, and lorry depots

11. Acceptance Criteria

11.1 Platform and Infrastructure

- MySQL boots locally from Docker and the application connects via environment variables.
- Alembic migrations create the full schema on a fresh database without errors.
- Seeds populate all three deterministic MVP scenarios.
- The demo can be fully reset and replayed on one laptop.

11.2 Ingestion

- Each snapshot reader supports latest and historical snapshot fetches.
- ETA refresh stores new ETA snapshots on each invocation.
- Manifest comes from manifest_reader — not manual upload.

11.3 Engines

- M1, M2, and M3 never mutate stock data. All three engines execute strictly on-demand via planner “Generate Plan” triggers — no background polling. All three engines consume Effective Stock (Physical − Reserved for warehouse; Physical + In-Transit for DC) to prevent ghost inventory double-allocation.
- Orchestration builds correct input contracts for M1, M2, and M3 from stored snapshots.
- Stub outputs match the intended engine contracts.
- Real engines can replace stubs without changing API routes or frontend screens.

11.4 Planner Flow

- Planner can edit draft plans before approval.
- Planner approval freezes a plan version, immediately writes reservation records and in-transit allocations to the primary DB, and prevents ghost inventory in subsequent planning cycles. Planner override edits must pass Math-Bound Validation (capacity, Effective Warehouse Stock, reefer compatibility) before the plan can be frozen.
- Override creates a new draft version linked to the original.
- Approved plan versions are immutable — the system rejects mutating them.

11.5 Demo State

- simulate_vessel_arrival.py increments physical warehouse DB stock and drops the manifest. simulate_lorry_arrival.py increments physical DC DB stock, removes warehouse reservations, and closes the active transfer. Subsequent “Generate Plan” triggers reflect updated Effective Stock in all engine inputs.
- Demo-state views show reservation, in-transit, and completed states accurately.

11.6 Frontend

- Dashboard, Inputs, M1 Priorities, M2 Requests, M3 Dispatch, History, Demo State, and Reports all render against local MySQL-backed data.
- All views work against both stub outputs and real engine outputs.

12. Delivery Sequence

Day	Focus	Key Deliverables
1	Freeze and scaffold	Contracts, rules, repo structure, folder skeleton
2	Platform foundation	MySQL setup, migrations, all 5 snapshot readers, ETA provider, seed data and route graph
3	M1 and stubs	M1 engine, engine stubs for M2 and M3, planner priority view
4	M2	M2 engine, DC request views
5	M3	M3 optimizer and ranker, candidate plan exposure
6	Planner flow and demo state	Approve, reject, override flow; demo reservation and arrival simulation
7	Harden and document	Demo scenario hardening, audit trail, reports, documentation

12.1 Team Responsibilities

Developer	Responsibility
Developer 1	Platform Data and Demo Scripts — build Effective Stock readers (Physical − Reserved, Physical + In-Transit), build Math-Bound Validation engine for planner overrides, build CLI simulation scripts (simulate_vessel_arrival.py, simulate_lorry_arrival.py), expose demo audit views
Developer 2	Frontend and Planner Flow — all console views, API orchestration wiring, reporting views
Developer 3	M1 and M2 — training, inference, score output, request generation; both isolated from planner and demo-state
Developer 4	M3 — optimizer, ranker, candidate plan generation, planner-editable draft contract; isolated from demo-state

13. Appendix

13.1 Environment Variables

Variable	Purpose
MYSQL_HOST	Hostname or container name of the MySQL service
MYSQL_PORT	MySQL port — default 3306
MYSQL_DATABASE	Database name — control_tower_mvp
MYSQL_USER	Application database user
MYSQL_PASSWORD	Application user password
MYSQL_ROOT_PASSWORD	MySQL root password — separate from application user

13.2 Developer Commands

Command	Action
docker compose up -d mysql	Start the MySQL container
alembic upgrade head	Run all pending migrations
python seed.py	Seed master data and demo scenarios
uvicorn main:app --reload	Start the FastAPI backend
npm run dev	Start the Next.js frontend
POST /api/v1/mock/eta/refresh	Manually trigger an ETA refresh
python scripts/simulate_vessel_arrival.py	Reads active manifests, increments physical warehouse DB stock, drops manifest. Run simulate_lorry_arrival.py to close transfers and increment DC DB stock.
python reset_demo.py	Full reset: drop and reseed demo scenarios

13.3 Engine Contract Summary

M1 Input Contract

- manifest_lines — list of shipment-SKU lines from the latest manifest snapshot
- effective_warehouse_stock — computed as Physical Warehouse Stock minus Reserved DB Stockactive_m2_requests — current open DC requests
- sku_metadata — master SKU data including reefer_required and capacity_unit
- eta_timing — ETA snapshot data for timing calculations

M1 Output Contract

- priority_score — numeric score per shipment-SKU line
- priority_band — categorical band (Critical, High, Medium, Low)

- reefer_required — boolean flag per line
- sku_summary — aggregated view of priority per SKU for dashboard display

M2 Input Contract

- effective_dc_stock — Effective DC Stock per DC per SKU (Physical DB DC Stock + In-Transit DB DC Stock); prevents ghost inventory double-allocation
- sales_forecast_48h — 48-hour forward sales forecast derived from rolling sales history; used to generate urgency and required_by values

M2 Output Contract

- dc_request — DC ID and SKU ID requiring restocking
- urgency — categorical urgency level
- required_by — ISO 8601 date by which stock must arrive

M3 Input Contract

- m1_output — full M1 scored and ranked lines
- m2_output — all active DC requests
- warehouse_stock_snapshot — current warehouse stock
- lorry_state_snapshot — current lorry availability and positions
- route_graph — fixed node-edge graph with travel times and costs

M3 Output Contract

- candidate_plans — list of feasible dispatch plans
- ranked_best_plan — the highest-ranked plan per the ranker model
- draft_plan — planner-editable version of the best plan with lorry, stop, and quantity fields